Chess Agent Competition: Guidelines for Competitors

1. Permitted approaches and originality

Teams may build their agent using any approach, including classical search, hand-crafted or tuned evaluation functions, neural networks, or any combination of these.

The fundamental rule of the competition is that **all of the agent's chess intelligence must be the team's own work**. Specifically:

- The agent's move selection, search and evaluation must be written by the team.
- Any neural network must be trained by the team, using data the team has collected or generated.
- The agent must not use, call, wrap or copy any existing chess engine, including Stockfish, Leela Chess Zero, Komodo or any other engine.
- The agent must not use neural network weights, evaluation parameters or training data produced by another engine. For example, positions scored by Stockfish may not be used to train or tune the agent.
- Code from tutorials, books or open-source projects may be studied for learning purposes, but any code adapted from such sources must be declared in the submission's README, and the core search and evaluation must not be copied.

2. Permitted libraries and resources

Libraries that implement the rules of chess are treated as utilities and may be used freely. Libraries or resources that judge positions or select moves are not permitted.

Resource	Status
Chess rules libraries (e.g. python-chess) for move generation, legality checks, board representation and UCI handling	Permitted
General-purpose libraries (e.g. NumPy, PyTorch, TensorFlow, ONNX Runtime)	Permitted

Resource	Status
Human game databases (e.g. the Lichess open database) for training or tuning	Permitted
Data generated by the team's own agent (e.g. self-play games or positions scored by the agent)	Permitted
Any existing chess engine, its code, weights or analysis	Prohibited
Opening books	Prohibited
Endgame tablebases (e.g. Syzygy)	Prohibited
Any external service called during play, including online APIs and language models	Prohibited

Opening books and endgame tablebases are excluded so that every move in a game is the product of the agent's own reasoning rather than a stored lookup.

3. Technical requirements

Every agent must operate as a UCI (Universal Chess Interface) engine. Agents are connected to Lichess through lichess-bot, an open-source bridge that relays each game to the agent using UCI, so an agent that meets the requirements below needs no Lichess-specific code.

1. **UCI compliance:** the agent must respond correctly to the commands `uci`, `isready`, `ucinewgame`, `position` (both `startpos` and `fen`, with moves), `go` (including `wtime`, `btime`, `winc` and `binc`) and `quit`, and must reply to each `go` command with a legal `bestmove`.
2. **Single launch command:** the agent must start with one command run from its own folder, for example `python main.py` or `./engine`.
3. **Clean output:** the agent must write only valid UCI responses to standard output. Debug information must be written to standard error or a log file.
4. **Relative paths:** any model, weights or data file must be loaded using a path relative to the agent's folder.
5. **Languages:** any programming language may be used, provided the agent can be run through lichess-bot using its launch command. Compiled agents must be submitted as a ready-to-run executable together with their source code.

6. **Python agents:** must state the Python version used in the README and list all dependencies in a `requirements.txt` file.

4. Time control and game rules

All tournament games are played at a time control of **3 minutes per side plus a 2-second increment per move**. Agents are responsible for managing their own time using the clock information sent with each `go` command.

A game is scored as a loss for an agent that:

- exceeds its time limit;
- plays an illegal move or sends a malformed response;
- crashes, stops responding or fails to start; or

Games are governed by Lichess's rules, which enforce checkmate, stalemate and the standard draw rules (threefold repetition, the fifty-move rule and insufficient material) automatically. An agent that fails to move loses on time under the Lichess clock. If a game is aborted before any moves are played because of a technical fault on the organisers' side, it will be replayed.

5. Setting up the Lichess bot account

Each team must create one Lichess bot account for its agent and run the bot on its own hardware throughout the competition. Bot accounts can only be used by programs through the Bot API and are clearly marked as BOT on Lichess.

Creating the account

1. **Create a new Lichess account** dedicated to the agent, with a username that identifies the team. The account must never have played a game, as only unused accounts can become bot accounts.
2. **Create an API access token.** While logged in to the bot account, go to Preferences → API access tokens and create a token with only the *Play games with the bot API* (`bot:play`) permission selected. Lichess displays the token only once, so it must be copied and stored securely.
3. **Install lichess-bot.** Download the official bridge from github.com/lichess-bot-devs/lichess-bot, install its requirements, and enter the token and the agent's launch file in its `config.yml` file, with the protocol set to UCI.

4. **Upgrade the account to a bot account** by running `python lichess-bot.py -u`. This step is permanent: the account can never again be used by a human player.
5. **Test the connection.** Start lichess-bot and challenge the bot to a casual game from a personal account to confirm that the agent plays correctly.
6. **Create a challenge token.** Create a second API access token on the bot account with only the *Create, accept, decline challenges* (`challenge:write`) permission selected. This token is submitted to the organisers on the day of the competition (see Section 6) and is used solely to start competition games; it does not allow the organisers to make moves for the bot. The `bot:play` token used by lichess-bot must remain private to the team.

What is permitted and what is not

The following settings are required in lichess-bot's `config.yml`:

- Accept standard chess only, at the competition time controls (3+2 and 10+5).
- Accept casual (unrated) games only.
- During the competition, accept challenges only from the organisers' accounts, competing bots and chess club members taking part in the final stage.
- Keep automatic matchmaking disabled.

The following are prohibited:

- **Online move sources:** lichess-bot's `online_moves` features, including Lichess cloud analysis, the Lichess opening explorer, ChessDB and online endgame tablebases, must be disabled, as these supply moves not chosen by the agent.
- **Local books and tablebases:** lichess-bot's opening book (polyglot) and tablebase options must be disabled, consistent with Section 2.
- **Manual play:** no person may make moves for the bot account by any means.
- **Misconduct:** chat messages must remain courteous, and the account must comply with Lichess's Terms of Service at all times.

Resigning or offering draws based on the agent's own evaluation is permitted.

Running the bot during the competition

Teams are responsible for hosting their own bot.

- The organisers announce the start time of each round in advance and start every game themselves, using the challenge tokens provided by the teams. No action is required from team members once their bot is online.
- Each bot must be online, running lichess-bot with the submitted agent, from the announced start time until all of its games in that round are complete.
- The bot must accept competition challenges automatically. Challenges must not be declined or left unanswered.
- A bot that is not online and accepting challenges within 10 minutes of the announced start time forfeits its game.
- If a bot disconnects during a game, the Lichess clock continues to run, and the game is decided under Lichess's rules.

6. Submission format

On the day of the competition, each team submits the following through the channel specified by the organisers. Late submissions will not be accepted.

- **GitHub repository:** a link to the team's repository, which must be accessible to the organisers.
- **Bot username:** the Lichess username of the team's bot account.
- **Challenge token:** the `challenge:write` token described in Section 5. T

The repository must be structured as follows:

```
team-name/
  engine/          launch file, executable and model files
  source/          complete source code, including training code
  README.md        launch command, approach summary, declared sources
  requirements.txt  (Python agents only)
```

The README must state the exact launch command and the Lichess bot username, give a brief description of the agent's approach, and declare any code adapted from external sources as required by Section 1.

Before submitting, each team must confirm that its agent meets the following checklist:

☐ Starts with the stated launch command and plays through lichess-bot without errors.
☐ Responds correctly to all UCI commands listed in Section 3.
☐ Manages its time correctly at the official time control.

- ☐ Contains no prohibited libraries, engines, opening books or tablebases.

7. Knockout stage

The knockout stage is played in rounds on Lichess, with the organisers issuing each challenge between the paired bots at the time control set out in Section 4.

1. **Random pairing:** at the start of each round, the remaining agents are paired at random.
2. **Single game:** each pairing plays one game, with colours assigned at random.
3. **Elimination:** the winner advances to the next round and the loser is eliminated.
4. **Drawn games:** if the game is drawn, it is replayed with colours reversed. If the replay is also drawn, a deciding game is played with colours assigned at random.
5. **Finalists:** rounds continue until three agents remain. These three agents are declared the winners of the competition.

All games are publicly viewable on Lichess, and the results of each round will be published with links to the games.

8. Final stage: winners against the chess club

The three winners of the knockout stage play against members of the chess club, using the same bot accounts on Lichess.

- Chess club members play from their own Lichess accounts, and games are arranged by the organisers as challenges to the bots.
- Games are casual and played at a rapid time control of **10 minutes per side plus a 5-second increment per move**.
- Colours are assigned at random, and the number of games each bot plays will be announced by the organisers.
- Agents must use exactly the version submitted for the knockout stage, with no modifications.
- Results are recorded and published, but do not change the three winners decided in the knockout stage.

9. Fair play and disqualification

The organisers will review the source code of every submission, with particular attention to the highest-ranked agents. Teams may be asked to explain their approach, demonstrate their training process or rebuild their agent from the submitted source code.

No human intervention is permitted once a tournament game has begun. Teams may not alter, restart or advise their agent during play.

An agent will be disqualified, and its results removed from the tournament, if it:

- uses or copies an existing chess engine, its weights or its analysis;
- uses an opening book, endgame tablebase or any other prohibited resource;
- accesses the internet or any external service during play, including lichess-bot's online move sources;
- contains undeclared code copied from external sources in its core search or evaluation;
- deliberately interferes with other agents, other bot accounts or the running of the competition;
- has moves made for it by a person, or breaches Lichess's Terms of Service; or
- cannot be reproduced from the submitted source code.